Universidad Iberoamericana León

Desarrollo de aplicaciones y servicios virtuales

RutaMx: persistencia de datos y diseño mínimo de base de datos

Práctica: Persistencia y diseño mínimo de base de datos. Apartado nuevo de la versión 2.0 (Descubrir y Planear con Ayuda IA).

Integrante	Rol principal (documento v2.0)
Roberto Valderrama	Arquitectura / diseño y análisis
Angel Mata	Operación de IA y validación técnica

Fecha: 15 de octubre de 2026. Estado: documento de diseño; no incluye implementación ni cambios en la base real. Base documental: Documento de requerimientos v2.0 (8 de octubre de 2026) y Diseño modular y arquitectura inicial (módulos M-01 a M-11).

1. Alcance y respeto a la versión 1

RutaMx tiene dos versiones. La **versión 1** (la app actual: formulario de ruta, sugerencias, cuentas, guardado de rutas, catálogo de destinos) está terminada y **no se modifica**. Esta práctica cubre solo el **apartado nuevo**: las ventanas **Descubrir** y **Planear con Ayuda IA**, que se construyen en `paginas_nav_bar/` sobre la infraestructura que ya existe (autenticación y persistencia), tal como pide el diseño modular: reutilizar primero, crear solo lo que falta.

Lo que la v1 ya persiste y se reutiliza tal cual: PostgreSQL en Supabase vía SQLAlchemy, con las tablas `estados`, `destinos`, `usuarios` (nombre + apellido + PIN con hash) y `rutas_guardadas` (nombre, origen, destino, intereses, resumen JSONB, paradas JSONB, usuario_id).

2. Paso 1: decisión de persistencia

Pregunta	Respuesta del equipo
¿El proyecto requiere guardar información después de cerrar el programa?	Sí. Las rutas guardadas deben recuperarse después (RF-38, RF-39, RF-40) y el catálogo de rutas públicas debe existir antes de que cualquier usuario entre a Descubrir (RF-01).
¿Qué información se guardará?	Rutas (con sus paradas ordenadas), su visibilidad privada o pública, su dueño y autor, y de qué ruta pública se copiaron. No se guarda la conversación del chat (D-09).
¿Se usará base de datos?	Sí.
¿Qué alternativa se usará si no hay base de datos?	No aplica para las rutas. Lo temporal (la propuesta de la IA antes de guardar, el estado del chat) vive solo en memoria de la sesión.
¿Qué tipo de base de datos se usará?	Relacional: la misma PostgreSQL (Supabase) que ya usa la v1. No se abre una segunda base.
¿Qué módulo administrará los datos?	M-09 Repositorio de rutas. Solo lo llama M-05 Gestor de rutas; ninguna interfaz ni la IA tocan la base directamente.
¿Qué requerimientos justifican esta decisión?	RF-01 a RF-10 (catálogo público), RF-13 a RF-16 (copiar), RF-38 a RF-42 (guardar), RF-40 y RF-44 (consultar y recuperar), RNF-11 (aislamiento entre usuarios), RNF-16 y RNF-22 (capas y lógica centralizada), decisiones D-08 a D-11.

3. Paso 2: ruta elegida y justificación

Ruta B: el proyecto sí requiere base de datos. Cumple varios de los criterios de la explicación del tema:

- Guarda registros que se consultan después (rutas guardadas y catálogo público).
- Tiene usuarios y permisos: solo el dueño modifica una ruta y solo quien inició sesión guarda o copia (RF-15, RF-41, RNF-11).
- Hay relaciones entre datos: usuario con rutas, y una ruta copiada con su ruta original.
- Hay que actualizar información (guardar de nuevo actualiza la misma ruta, D-11).
- Varios módulos dependen de las mismas rutas (Descubrir, Planear con IA y rutas guardadas comparten M-05 y M-09).
- Guardar y copiar deben ser atómicos: o se completa o no se crea nada (RF-16, RF-42). Una base transaccional lo garantiza; un archivo JSON o CSV no, sobre todo con varios usuarios a la vez.

Alternativas descartadas. CSV/JSON: no dan atomicidad ni control de acceso por usuario. SQLite: obligaría a mantener una segunda base, distinta de la que ya guarda cuentas y rutas. MongoDB: las rutas son datos con relaciones claras (usuario, ruta original) y la v1 ya usa SQL. Con PostgreSQL se reutiliza lo que existe y se cumple el criterio de integración del diseño modular.

4. Diseño mínimo de base de datos

Entidades del modelo conceptual (diseño modular, sección 8): **Ruta** y **Lugar**. Decisión: **Ruta** se guarda en la tabla existente `rutas_guardadas`, a la que se le agregan columnas (opción elegida por el equipo). **Lugar** no es tabla nueva: cada parada va dentro de la ruta (columna `paradas`, JSONB ordenado) con nombre, coordenadas e identificador geográfico (D-03), así guardar una ruta completa es una sola escritura y la ruta estructurada sigue siendo la fuente única de verdad (D-08).

Entidad / tabla	Atributos principales	Llave primaria	Relaciones	Requerimiento
usuarios (existe, sin cambios)	nombre, apellido, clave_normalizada, pin_hash, creado_en	id	1 usuario : N rutas (dueño); 1 usuario : N rutas (autor)	RF-15, RF-39, RF-41, RNF-10, RNF-11
rutas_guardadas (existe; se extiende)	Existentes: nombre, origen, destino, intereses, resumen, paradas, usuario_id, creado_en, presupuesto. Nuevas: descripcion, duracion_dias, categoria, region, visibilidad, ruta_origen_id, autor_id	id	usuario_id FK a usuarios (dueño); autor_id FK a usuarios (creador original); ruta_origen_id FK a rutas_guardadas (de cuál ruta pública se copió)	RF-01 a RF-10, RF-13 a RF-16, RF-38 a RF-42, RF-44, D-08, D-10, D-11
destinos, estados (existen, sin cambios)	Catálogo de lugares de la v1	id	destinos.estado_id FK a estados	Apoyo a RF-11, RF-12 (no se tocan)

Las columnas nuevas son nulas o tienen valor por defecto, por lo que **las filas que ya existen de la v1 siguen siendo válidas** (quedan privadas, sin ruta original).

Diccionario de datos básico (campos nuevos de rutas_guardadas)

Campo	Tipo de dato	Descripción	Obligatorio	Ejemplo (ficticio)
descripcion	TEXT	Texto del detalle de la ruta (RF-10).	No	Ruta corta de fin de semana con paradas de comida.
duracion_dias	INTEGER (mayor que 0)	Duración estimada; sirve para el filtro y el detalle (RF-05, RF-10; dato mínimo de planeación D-02).	No (sí para publicar)	3

Campo	Tipo de dato	Descripción	Obligatorio	Ejemplo (ficticio)
categoria	TEXT	Categoría para el filtro. Los valores válidos salen de los datos reales del catálogo (P-03, pendiente).	No	por definir (P-03)
region	TEXT	Región para el filtro. Valores por definir con datos reales (P-03).	No	por definir (P-03)
visibilidad	TEXT: 'privada' o 'publica'	Estado de la ruta (D-10). Solo las públicas aparecen en Descubrir.	Sí (por defecto 'privada')	privada
ruta_origen_id	INTEGER, FK a rutas_guardadas.id	Ruta pública de la que se copió. Nula si la ruta no es copia. La original nunca se modifica (RF-14).	No	12
autor_id	INTEGER, FK a usuarios.id	Quien creó la ruta originalmente; no cambia al copiarla. El dueño (usuario_id) sí cambia.	No	4
Campos existentes que se reutilizan	nombre, origen, destino, intereses (ARRAY), paradas (JSONB), resumen (JSONB), usuario_id, creado_en	Tal como los define la v1. Cada parada del JSON debe traer identificador y coordenadas válidas (D-03).	origen, destino, nombre y paradas: sí	Origen: Leon, Guanajuato (ficticio)

Reglas de integridad: visibilidad solo acepta 'privada' o 'publica' (restricción CHECK); duracion_dias es mayor que 0; ruta_origen_id usa ON DELETE SET NULL para que borrar una ruta original no destruya las copias; una ruta pública debe tener origen, destino, duración y paradas completas antes de publicarse (RF-10).

Operaciones de datos

Operación	Descripción	Módulo responsable
Crear	Guardar una ruta generada con IA como nueva fila: dueño y autor son la cuenta autenticada, visibilidad privada (RF-38, RF-39). Copiar una ruta pública: nueva fila con los datos de la original, dueño = quien copia, autor = autor original, ruta_origen_id = original; se hace en una sola transacción, y si falla no queda copia incompleta (RF-14, RF-16, RF-42).	M-05 decide y valida; M-09 escribe; M-10 comprueba sesión (RF-15, RF-41).
Consultar	Listar rutas públicas con filtros de duración, categoría y región (RF-05, RF-06); detalle de una ruta (RF-09, RF-10); listar solo las rutas del usuario autenticado (RF-40, ya existe como listar_rutas). Si falla la lectura se muestra error sin tocar datos (RF-44).	M-02 arma la consulta del catálogo; M-09 lee.
Actualizar	Guardar de nuevo una ruta ya guardada actualiza esa misma fila, sin historial (D-11); si falla, queda intacta la versión anterior. Solo el dueño puede hacerlo (RNF-11). Cambiar visibilidad a pública lo controla el personal de RutaMx (D-10); el mecanismo está pendiente (P-01).	M-05 aplica el cambio de forma atómica; M-09 escribe; M-10 verifica propiedad.
Eliminar	Ningún RF del MVP pide borrar una ruta completa; por eso no se implementa. Sí existe eliminar paradas de una ruta en edición, que es una actualización del campo paradas (RF-27). Si más adelante se agrega, debe pasar por M-10 (solo el dueño, PNF-11).	M-05 / M-09 (si se agrega).

5. Módulo responsable del manejo de datos

M-09 Repositorio de rutas (capa de datos). Es el único que habla con la base para las rutas del apartado nuevo; el diseño modular lo define como el módulo que "lee y escribe rutas y su asociación con la cuenta, y garantiza que no se registren datos incompletos".

- **Quién lo usa:** solo M-05 Gestor de rutas (y M-02 para la consulta del catálogo). Las interfaces (M-01, M-03, M-06), el orquestador (M-04) y el adaptador de IA (M-07) nunca acceden a la base.

- **Qué no hace:** no valida reglas de negocio (eso es M-05), no revisa sesiones (M-10) y no habla con la IA ni con mapas.
- **Reutilización:** usa la conexión y la sesión de SQLAlchemy que ya configura la v1 y el modelo de rutas existente, ampliado con las columnas nuevas. No crea una segunda conexión.
- **Atomicidad:** cada guardar o copiar es una transacción: se confirma completa o se revierte (RF-16, RF-42).
- **Ubicación propuesta:** dentro de `paginas_nav_bar/`, como parte de los componentes del apartado nuevo, y no como una carpeta raíz nueva (el diseño modular dice que los once módulos no se vuelven once carpetas de primer nivel). El nombre sigue la convención del proyecto en español (por ejemplo `rutas_repositorio.py`).
- **Pruebas:** con una base de prueba aislada y datos ficticios, sin tocar la base real ni depender del .env real.

6. Qué no se persiste

- La conversación del chat y los mensajes de la IA (D-09): solo se guarda la ruta resultante.
- La propuesta de ruta que aún no se guardó: vive en la sesión; la ruta estructurada es la fuente de verdad y el chat, el itinerario y el mapa se derivan de ella (D-08).
- Historial o versiones de una ruta (D-11).
- Credenciales de IA o de mapas: van en variables de entorno, nunca en la base ni en el repositorio.

7. Riesgos, pendientes y puntos que requieren aprobación

Punto	Detalle	Control o siguiente paso
Se extiende una tabla de la v1	Agregar columnas a rutas_guardadas toca una tabla de la versión 1, que el equipo había acordado no modificar. El equipo eligió esta opción por simplicidad.	Las columnas son opcionales o con valor por defecto, y no cambia ningún comportamiento actual. Aun así, el cambio real en la base y en models.py espera confirmación explícita de Angel.
Migración	SQLAlchemy create_all no modifica tablas que ya existen: hay que aplicar ALTER TABLE en Supabase.	Escribir un script de migración revisable, probarlo primero en una copia, y respaldar antes.
P-01	No está definido cómo una ruta pasa a pública (D-10 dice que lo controla el personal de RutaMx).	Confirmar con el cliente. La columna visibilidad ya soporta cualquiera de las opciones; no se inventa pantalla ni flujo.
P-03	Los valores de categoría y región no están definidos.	Obtenerlos de los datos reales del catálogo, no inventarlos.
Paradas en JSONB	No se pueden consultar paradas individuales con facilidad.	Aceptable para el MVP: ningún RF consulta paradas sueltas. Si hiciera falta, se normaliza a una tabla paradas.
Rutas públicas iniciales	Hace falta saber quién crea el contenido que se mostrará en Descubrir.	Por definir con el cliente junto con P-01. Cualquier dato de prueba será ficticio.
Aislamiento entre usuarios	Un error de consulta podría mostrar rutas ajenas.	Toda consulta de rutas propias filtra por usuario_id y se prueba con el caso PNF-11.

8. Trazabilidad requerimiento y decisión

Requerimiento / decisión	Cómo lo atiende el diseño
RF-01 a RF-10, D-01	Catálogo público: visibilidad, duracion_dias, categoria, region, descripcion; consulta por M-02 y M-09.
RF-13 a RF-16	Copia como nueva fila con ruta_origen_id, en transacción única; la original no cambia.

Requerimiento / decisión	Cómo lo atiende el diseño
RF-38 a RF-42, D-11	Guardar asocia a la cuenta; guardar de nuevo actualiza la misma fila; el fallo no deja datos incompletos.
RF-40, RF-44, D-07	Listar rutas del usuario; error de lectura sin modificar otros datos.
RNF-11, RF-15, RF-41	Dueño en usuario_id; M-10 verifica sesión y propiedad antes de escribir.
RNF-16, RNF-20, RNF-22	La base solo se accede desde M-09; Descubrir y Planear no se llaman entre sí; la lógica de rutas es solo de M-05.
D-08, D-09	Se guarda la ruta estructurada; no se guarda el chat.
D-10, P-01	Columna visibilidad; mecanismo de publicación pendiente.

9. Bitácora de uso de IA

Se registra solo lo que pasó en esta sesión. Herramienta: Claude Code (modelo Sonnet 5.5), operado por Angel Mata. Las celdas marcadas como pendientes no se han hecho todavía.

Fase	Prompt usado	Resultado de la IA	Qué aceptamos	Qué modificamos	Responsable
Decisión de persistencia	Se entregaron la práctica y la explicación del tema y se pidió hacerla respetando el proyecto y sus versiones, preguntando antes de actuar.	Leyó la práctica, los requerimientos v2.0, el diseño modular y el código de la v1. Concluyó que sí hay persistencia (PostgreSQL de Supabase, ya existente) y preguntó tres dudas antes de escribir.	Ruta B con PostgreSQL reutilizado; M-09 como módulo; sin código en esta entrega.	Ninguna.	Angel Mata
Diseño de BD	Respuestas a las dudas: solo documento; agregar columnas a rutas_guardadas; formato PDF.	Diseño mínimo: columnas nuevas (visibilidad, ruta_origen_id, autor_id, duracion_dias, categoria, region, descripcion), diccionario y CRUD.	Extender rutas_guardadas en lugar de crear tablas nuevas.	Angel eligió extender la tabla de la v1 en vez de la tabla ligada recomendada por la IA; el cambio real queda pendiente de su confirmación.	Angel Mata
Plan del módulo	Mismo diseño modular de la práctica anterior.	Definió M-09 como único acceso a datos, llamado solo por M-05, con transacciones.	Pendiente de revisión de Roberto.	Pendiente.	Equipo
Implementación	No aplica.	No se escribió código ni se tocó la base ni la v1.	No aplica.	No aplica.	No aplica
Revisión	Pendiente.	Pendiente: revisión de Roberto y comparación con el documento de requerimientos.	Pendiente.	Pendiente.	Roberto Valderrama
Pruebas	Pendiente.	Sin pruebas todavía, porque no hay módulo implementado.	Pendiente.	Pendiente.	Equipo

10. Preguntas de cierre

Pregunta	Respuesta
1. ¿Qué módulo implementaron primero y por qué?	Todavía ninguno. Esta entrega es solo diseño. El candidato es M-09 Repositorio de rutas, porque M-05, M-02 y el guardado dependen de él.
2. ¿Qué requerimiento atiende ese módulo?	RF-14, RF-16, RF-38 a RF-40, RF-42 y RF-44 (ver la tabla del módulo M-09 en el diseño modular).
3. ¿El proyecto requiere base de datos? ¿Por qué?	Sí: hay usuarios y permisos, relaciones entre datos, actualización y consulta de registros, y operaciones que deben ser atómicas (sección 3).
4. ¿Qué módulo será responsable de manejar los datos?	M-09 Repositorio de rutas, usado solo por M-05.
5. ¿Qué parte del código generó Codex?	Ninguna en esta entrega: no hay código. El documento se redactó con Claude Code; el equipo debe indicar aquí si Codex participó en otra parte.
6. ¿Qué revisó o corrigió el equipo?	Angel decidió el alcance, el formato y extender rutas_guardadas. La revisión técnica de Roberto sigue pendiente.

Pregunta	Respuesta
7. ¿Qué prueba manual demuestra que el módulo funciona?	Todavía no hay módulo. Prueba propuesta: guardar una ruta ficticia, copiarla con otra cuenta y comprobar que la original no cambia y que sin sesión no se crea nada.
8. ¿Qué quedó pendiente para la siguiente semana?	Revisión de Roberto, resolver P-01 y P-03, aprobar el cambio a la tabla de la v1 y elegir el primer módulo a implementar.

Tarea para la siguiente semana

- Revisar este diseño con Roberto y corregirlo si aparecen problemas.
- Confirmar P-01 y P-03 con el cliente (profesor).
- Aprobar de forma explícita el cambio de columnas en rutas_guardadas y preparar el script de migración.
- Escoger el segundo módulo a implementar y agregar la evidencia de commits.